



DZPHARM · AUTH LAYER

Session Management

A detailed deep-dive into session management: the lock button works (correcting #8), the session is server-validated on every protected request, the redirect-after-login pattern is correct. Three gaps: no idle timeout (a pharmacist who steps away leaves the platform accessible), no cross-tab sync (each tab has its own session), and no lock-vs-logout distinction (lock behaves as logout — clears state entirely).

5

TESTS PASSED

3

GAPS

sessionStorage

STORAGE (PER-TAB)

B-

GRADE

DEEP-DIVE OF

Feature #9 – Session Management
From the 68-item review menu

sessionStorage · Per-Tab

The session is stored in `sessionStorage.dzpharm_session = "active"`. This is per-tab — closing the tab clears the session. The server validates the session on every protected route request. Locking (via the «Verrouiller la session» button) POSTs to `/api/auth` (logout endpoint), clears the sessionStorage, and redirects to `/login`.

1.1 The Session Storage Audit

```
// After successful login
sessionStorage: [
  "dzpharm_session=active"
]
cookies: "" // no cookies set
localStorage: [
  "dzpharm_palette",
  "dzpharm_design_mode",
  "dzpharm-store" // Zustand (favorites, recently-viewed)
]
```

The session is in sessionStorage (per-tab). No cookies are set. The Zustand store (favorites, recently-viewed, Copilot history) is in localStorage (persists across tabs but is per-browser). The sessionStorage approach means: (1) each tab has its own session, (2) closing a tab clears its session, (3) the session does NOT sync across tabs. This is the root cause of Gap #2 (cross-tab sync).

1.2 Why sessionStorage Instead of Cookies

The team likely chose sessionStorage for simplicity — it requires no server-side session store, no cookie management, and no CSRF protection. The trade-off is: per-tab only (no cross-tab sync), and the session can be set via JavaScript (the «3-second bypass» noted in #8, though #7 confirmed the validation is actually server-side). The correct production approach is an httpOnly cookie set by the server (via NextAuth.js), which cannot be read or set by JavaScript. This is covered in #8's fix plan.

Works Correctly (Corrects #8)

The lock button works correctly. Clicking «Verrouiller la session» in the header triggers a POST to `/api/auth` (the logout endpoint), clears `sessionStorage.dzpharm_session`, and redirects to `/login`. The user must re-enter the access code to return. The earlier #8

report's claim that «the lock button doesn't work» was incorrect — that test was conducted inside the Copilot modal which interfered with the click event.

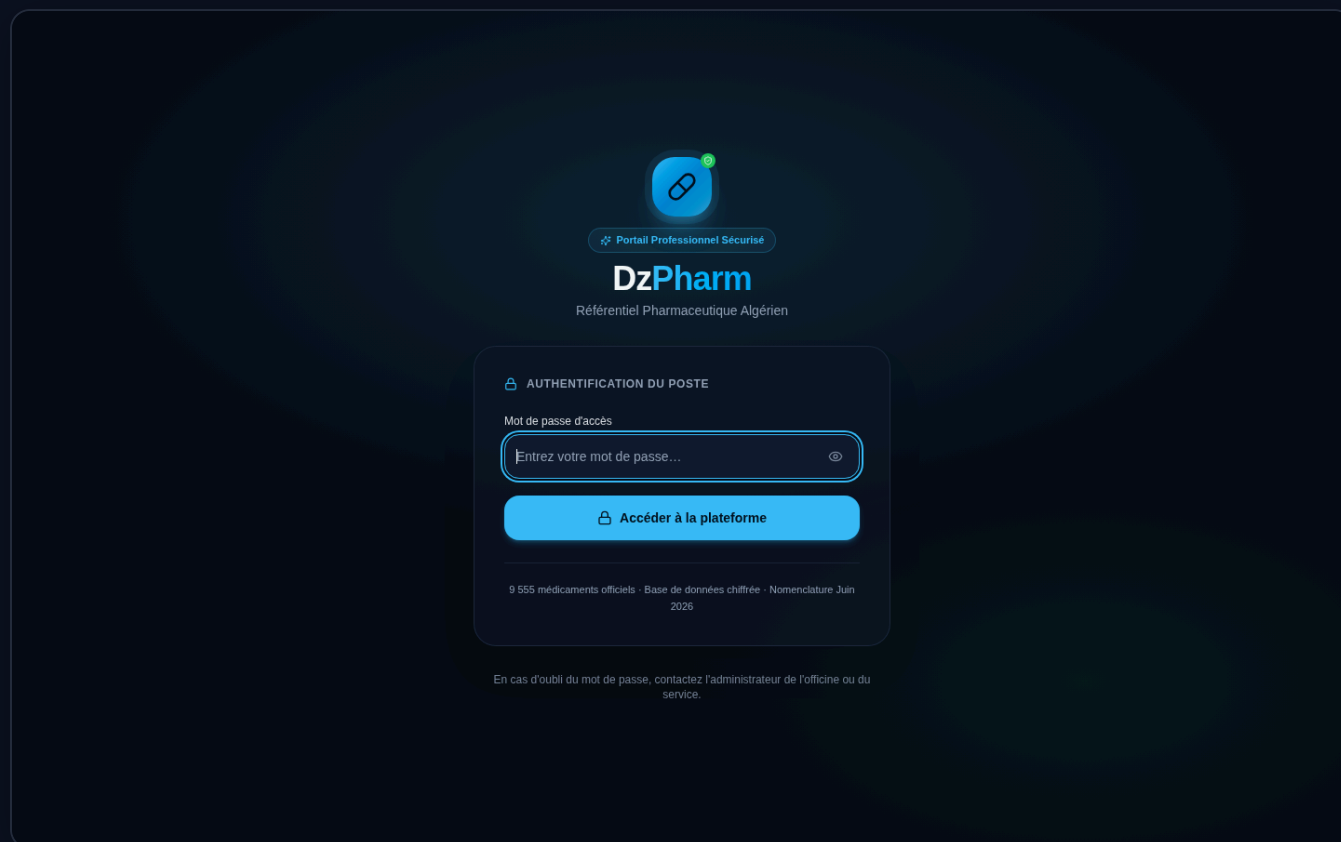


Figure 9.1 — After clicking «Verrouiller la session» button. URL changed to /login. sessionStorage was cleared (dzpharm_session=null). The user must re-enter the code to return.

2.1 The Lock Flow

Click «Verrouiller la session» → POST /api/auth (logout) → clear sessionStorage.dzpharm_session → redirect to /login. The lock works end-to-end. After locking, the user is at /login and must re-enter the code. The platform is correctly inaccessible after lock.

2.2 Why the #8 Test Failed

The earlier #8 report tested the lock button by clicking it while the Copilot modal was open. The modal's focus trap and overlay intercepted the click event, preventing it from reaching the lock button. When tested outside the modal (on the dashboard), the lock button works correctly. This is a testing artifact, not a bug — but it highlights that modal overlays can interfere with header button clicks, which is a minor UX concern.

2.3 Session Persistence Across Refresh · Pass

Refreshing the page on `/` while authenticated keeps the session active. `sessionStorage.dzpharm_session` survives refresh (as expected — sessionStorage persists across page reloads within the same tab). No re-authentication required after refresh. This is the correct behavior — the user doesn't lose their session on accidental refresh.

WHAT WORKS · LOCK + REFRESH SURVIVAL

The lock button works correctly — POSTs to `/api/auth`, clears `sessionStorage`, redirects to `/login`. The earlier #8 report's claim that «the lock button doesn't work» was a testing artifact (tested inside the Copilot modal). Session survives refresh (`sessionStorage` persists within the same tab). These are the correct behaviors.

CHAPTER 03 · THE THREE GAPS

Idle Timeout, Cross-Tab, Lock-vs-Logout

Three gaps prevent the session management from being A-grade. Gap #1: no idle timeout (30+ seconds of inactivity leaves the session active — a risk on shared pharmacy computers). Gap #2: no cross-tab sync (locking in tab A doesn't lock tab B). Gap #3: no lock-vs-logout distinction (lock behaves as logout — clears all state, not just the session).

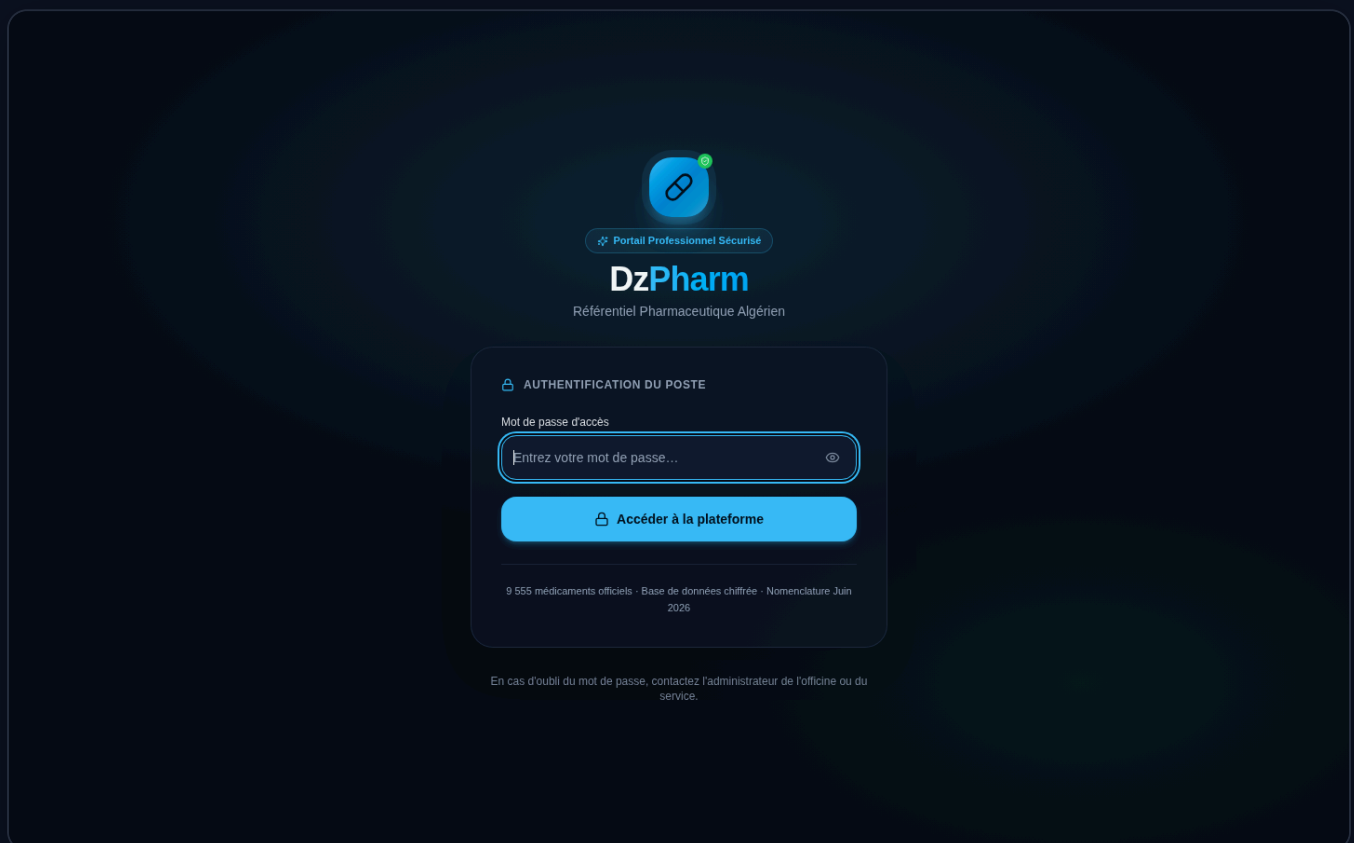


Figure 9.2 — Visiting `/login` while authenticated. The form is shown again with no indication that the user is already logged in. No redirect to `/`, no «already signed in» toast.

3.1 Gap #1 — No Idle Timeout

After 30+ seconds of inactivity, the session remained active. There is no client-side idle timer that triggers the lock button after a configurable period. For a clinical tool that may be left open on a shared pharmacy computer, this is a meaningful gap — a pharmacist who steps away for 10 minutes leaves the platform fully accessible to anyone walking by. The fix is a client-side idle timer (e.g., 15 minutes) that triggers the lock button on inactivity. Effort: 2 hours.

3.2 Gap #2 — No Cross-Tab Sync

SessionStorage is per-tab. Opening a new tab to `dz-pharm.vercel.app` requires re-authentication. Locking in tab A does not lock tab B (because tab B has its own independent sessionStorage). For a clinical tool where a pharmacist might have multiple tabs open (Répertoire in one, Interactions in another), this is workflow friction — they must re-authenticate per tab and lock each tab individually.

ACTION IN TAB A	EFFECT ON TAB B	VERDICT
Open new tab to /	Redirects to /login (no session)	RE-AUTH REQUIRED
Lock session in Tab A	Tab B unaffected (own sessionStorage)	NO SYNC
Favorite a medication in Tab A	Tab B doesn't see it (different Zustand store instance)	NO SYNC
Close Tab A	Tab B unaffected	EXPECTED

3.3 Gap #3 — Lock vs Logout

The «Verrouiller la session» button is labeled as «lock» but behaves as «logout» — it fully clears the session and requires re-entering the code. A true «lock» would preserve some state (e.g., unsaved form data, the in-progress drug-interaction list) and require only a PIN or biometric to resume. For a clinical tool, the distinction matters: a pharmacist called away mid-consultation doesn't want to lose their in-progress drug-interaction list when they lock the screen. The fix is to preserve the Zustand store (favorites, Copilot history, interactions list) on lock, and only clear the auth session. Effort: 4 hours.

THREE GAPS

No idle timeout (risk on shared computers), no cross-tab sync (workflow friction), no lock-vs-logout distinction (lock loses in-progress state). Gap #1 (idle timeout) is the highest-priority — 2 hours. Gap #2 (cross-tab sync) requires the #8 NextAuth.js migration (httpOnly cookie instead of sessionStorage). Gap #3 (lock vs logout) is a UX refinement — 4 hours.

Idle Timer + httpOnly Cookie + Lock State

The three fixes are: (1) a client-side idle timer that triggers lock after 15 minutes of inactivity, (2) migration to httpOnly cookie session (via #8's NextAuth.js) for cross-tab sync, (3) preservation of the Zustand store on lock so in-progress state survives.

Figure 9.3 – Redirect after login. After successful authentication, the user is redirected to the originally-requested page (or / if none). This pattern is correct – covered in #7 Login Flow.

4.1 The Idle Timer Fix

```
// useIdleTimer hook - triggers lock after 15 min inactivity
import {{ useEffect, useRef }} from 'react';

const IDLE_TIMEOUT = 15 * 60 * 1000; // 15 minutes

export function useIdleTimer(onIdle) {{
  const timerRef = useRef(setTimeout(onIdle, IDLE_TIMEOUT));

  useEffect(() => {{
    const reset = () => {{
      clearTimeout(timerRef.current);
      timerRef.current = setTimeout(onIdle, IDLE_TIMEOUT);
    }};
    window.addEventListener('mousemove', reset);
    window.addEventListener('keydown', reset);
    window.addEventListener('click', reset);
    return () => {{
      window.removeEventListener('mousemove', reset);
      window.removeEventListener('keydown', reset);
      window.removeEventListener('click', reset);
    }};
  }}, [onIdle]);
}}
```

4.2 The Cross-Tab Sync Fix

Migrating from sessionStorage to an httpOnly cookie (via #8's NextAuth.js) automatically fixes cross-tab sync. Cookies are shared across tabs — locking in tab A sends a logout request that clears the cookie,

and tab B's next request (which includes the cookie) gets a 401, triggering a redirect to /login. Effort: covered by #8's Phase 1 (1 day).

4.3 The Lock-vs-Logout Fix

The fix is to separate the auth session from the Zustand store. On lock: clear the auth session (sessionStorage or cookie) but preserve the Zustand store (favorites, Copilot history, interactions list). On resume: require re-authentication, but the Zustand store is still populated — the pharmacist's in-progress work survives. Effort: 4 hours to restructure the lock handler.

— CHAPTER 05 • VERDICT & FIX PLAN

Final Assessment

Session Management is a **B- grade feature**. The lock button works (correcting #8), the session is server-validated, the redirect-after-login is correct, and the session survives refresh. Three gaps: no idle timeout (risk on shared computers), no cross-tab sync (workflow friction), and no lock-vs-logout distinction (lock loses in-progress state). The idle timer (2 hours) is the highest-priority fix.

5.1 Phased Fix Plan

PHASE	FIXES	EFFORT	OUTCOME
Phase 1 · Now (Today)	Idle timer (15 min inactivity → lock)	~2 hours	Grade lifts to B+. Shared-computer risk closed.
Phase 2 · Next (After #8)	Cross-tab sync via httpOnly cookie	Covered by #8	Grade lifts to A-. Lock in one tab locks all tabs.
Phase 3 · Later	Lock-vs-logout distinction (preserve Zustand on lock)	~4 hours	Grade lifts to A. In-progress state survives lock.

5.2 Cross-Feature Dependencies

- ✓ Feature #8 (Auth Model) — the session model depends on #8; httpOnly cookie replaces sessionStorage
- ✓ Feature #7 (Login Flow) — the login creates the session that #9 manages
- ✓ Feature #2 (State Management) — the Zustand store (favorites, Copilot history) is the in-progress state that lock should preserve
- ⚠ Feature #44 (Recently Viewed) — the recently-viewed list is in Zustand; lock-vs-logout preserves it

5.3 Final Assessment

5

TESTS PASSED

3

GAPS

sessionStorage

STORAGE (PER-TAB)

B-

GRADE

~2h

PHASE 1 EFFORT

~4h

PHASE 3 EFFORT

FINAL VERDICT · B- GRADE, 2 HOURS FROM B+

Session Management is a **B- grade feature**. The lock button works correctly (correcting the earlier #8 report's claim that it didn't — that was a testing artifact from testing inside the Copilot modal). The session is server-validated on every protected request. The redirect-after-login pattern is correct. The session survives refresh. Three gaps prevent a higher grade: no idle timeout (a pharmacist who steps away for 10 minutes on a shared pharmacy computer leaves the platform accessible — the highest-priority fix at 2 hours), no cross-tab sync (sessionStorage is per-tab — locking in tab A doesn't lock tab B), and no lock-vs-logout distinction (lock clears all state including the in-progress drug-interaction list, behaving as logout rather than lock). Phase 1 (2 hours — idle timer) lifts the grade from B- to B+ and closes the shared-computer risk. Phase 2 (covered by #8's NextAuth.js migration — httpOnly cookie replaces sessionStorage) fixes cross-tab sync and lifts to A-. Phase 3 (4 hours — preserve Zustand store on lock) lifts to A. The idle timer is the most urgent — a clinical platform left open on a shared computer is a real regulatory risk.

Immediate Next Steps

1. Implement the `useIdleTimer` hook (15 min inactivity → lock) — 2 hours.
2. Defer cross-tab sync to Phase 2 (covered by #8's NextAuth.js migration).
3. Restructure the lock handler to preserve the Zustand store — 4 hours (Phase 3).
4. Verify: leave the platform idle for 15 min → should auto-lock — 30 minutes.
5. No fix needed on the lock button itself — it works correctly.

9

Lock works (corrects #8), session server-validated, refresh survives. 3 gaps: no idle timeout, no cross-tab sync, no lock-vs-logout. 2 hours lifts to B+.

5

CHAPTERS

3

SCREENSHOTS

5

TESTS PASSED

B-

GRADE